ASSIGNMENT ANSWERS

Introduction to Asymmetric Cryptography

Cryptography and Network Security

All 8 questions answered

Prepared in clear, student-friendly language with examples, formulas, diagrams and comparison tables.

Q1. Basic Principles of Asymmetric (Public-Key) Cryptography [10 Marks]

Asymmetric cryptography, also called public-key cryptography, is a method of securing information by using a mathematically related pair of keys: a **public key** and a **private key**. Unlike symmetric cryptography, the same secret key is not shared by both parties. The public key may be distributed openly, while the private key must be kept secret by its owner.

Public key: It is shared with other people. It can be used to encrypt information for its owner or to verify a digital signature made by that owner. **Private key:** It is kept confidential by its owner. It is used to decrypt information encrypted with the matching public key or to create a digital signature.

Confidentiality: If Alice wants to send a secret message to Bob, she encrypts it using Bob's public key. Only Bob's matching private key can decrypt it. Therefore, someone who intercepts the ciphertext should not be able to read the message.

Authentication and non-repudiation: Alice can create a digital signature using her private key. Bob checks the signature using Alice's public key. A valid signature helps confirm that the message was signed using the private key corresponding to Alice's public key and that the signed data has not been altered. This supports authentication, integrity and non-repudiation, provided Alice's private key remains under her control and her public key is reliably associated with her identity.

A single public-key system can therefore support different security goals, but encryption and signing are separate operations: encrypt to the recipient for confidentiality; sign with the sender's private key for authentication and integrity.

Labelled process diagram

```
                                CONFIDENTIALITY
        Alice (sender)   -- message + Bob's PUBLIC KEY --> Encryption
                                                |
                                                v
                          Ciphertext travels over an insecure network
                                                |
                                                v
        Bob (receiver) -- Bob's PRIVATE KEY --> Decryption --> Original message
```

```
                                SIGNATURE
  Alice -- hashes message and signs hash with Alice's PRIVATE KEY --> Digital signature
Bob -- uses Alice's PUBLIC KEY to verify signature and compares message hash --> Valid /
                                Invalid
```

Example: Alice sends Bob a confidential document by encrypting it with Bob's public key. She can also sign the document with her own private key. Bob decrypts it with his private key and verifies Alice's signature with her public key.

Q2. Difference Between Symmetric and Asymmetric Cryptography [8 Marks]

The main differences are shown below:

Heading	Symmetric cryptography	Asymmetric cryptography
Number of keys	One shared secret key is used for encryption and decryption.	A pair of mathematically related keys is used: public and private.
Key distribution	The secret key must be delivered securely to every communicating party.	The public key can be shared openly; the private key is kept secret. Public-key authenticity still needs verification.

Heading	Symmetric cryptography	Asymmetric cryptography
Speed	Generally very fast; suitable for large amounts of data.	Generally slower and more computationally expensive.
Key length	Typically shorter keys provide strong security, e.g. AES-128 or AES-256.	Usually much larger keys are needed for comparable security, e.g. RSA-2048 or RSA-3072.
Scalability for n users	If every pair needs a separate shared key, n(n–1)/2 keys may be needed.	Each user can have a key pair; public keys can be distributed, reducing pairwise secret-key distribution.
Typical applications	File encryption, disk encryption, bulk data transfer and secure sessions.	Key establishment, digital signatures, certificates and identity authentication.

For n users, the symmetric pairwise-key count is n(n–1)/2. For example, 10 users would require 45 distinct pairwise shared keys if each pair communicates privately. Public-key systems make distribution easier because each user can publish one public key, although certificates or another trusted method are needed to establish whose key it is.

Q3. Main Techniques and Building Blocks in Public-Key Cryptography

[10 Marks]

1. Trapdoor one-way functions: A one-way function is easy to compute in one direction but computationally difficult to reverse. A trapdoor is secret information that makes the reverse operation practical for an authorised person. RSA is based on the practical difficulty of factoring a product of two large primes; knowledge of the private key enables decryption or signing. Such functions form the mathematical basis of many public-key algorithms.

2. Diffie–Hellman key exchange: Diffie–Hellman allows two parties to establish a shared secret over a public network without sending the secret itself. The parties agree on public parameters, choose private values, exchange corresponding public values, and independently calculate the same shared secret. An eavesdropper sees the exchanged values but should find it computationally infeasible to derive the secret. Basic Diffie–Hellman does not authenticate the parties by itself, so it must be combined with authentication to prevent a man-in-the-middle attack.

3. Digital signatures: The sender creates a hash (fixed-length digest) of the message and signs it using their private signing key. The receiver uses the sender's public key to verify the signature. A valid signature provides evidence of origin, detects changes to the signed data, and can support non-repudiation. It does not encrypt the message; confidentiality requires a separate encryption mechanism.

4. Public-Key Infrastructure (PKI) and digital certificates: PKI is a framework of people, policies, software and trusted organisations used to manage public keys. A Certificate Authority (CA) issues a digital certificate that binds an identity, such as a website name, to a public key. The certificate is digitally signed by the CA. A receiver validates the certificate chain, validity period and other checks before trusting the public key. PKI helps prevent an attacker from simply substituting their own public key.

Together, these techniques enable secure key establishment, identity verification, message integrity and protected communication.

Q4. Historical Development of Public-Key Cryptography [10 Marks]

(a) The key distribution problem: Traditional symmetric encryption requires both parties to possess the same secret key before communicating. Securely delivering and managing that key becomes difficult when users are numerous or geographically separated. The need for a method to establish keys without first sharing a secret motivated public-key cryptography.

(b) Diffie and Hellman (1976): Whitfield Diffie and Martin Hellman published the influential paper “New Directions in Cryptography” in 1976. It introduced the public-key concept and described a method for key exchange over an

insecure channel, now known as Diffie–Hellman key exchange. It changed the way cryptographers approached key distribution.

(c) RSA (1977): Ron Rivest, Adi Shamir and Leonard Adleman developed RSA in 1977. RSA uses a public/private key pair and relies on the difficulty of factoring a large composite number into its prime factors. It became one of the best-known public-key algorithms and has been used for encryption and digital signatures.

(d) Earlier classified GCHQ work: Related ideas were independently developed earlier at the UK's Government Communications Headquarters (GCHQ). James Ellis developed the concept of non-secret encryption; Clifford Cocks devised an RSA-like system; and Malcolm Williamson developed a key-agreement method related to Diffie–Hellman. This work remained classified for years and became publicly known later.

(e) ElGamal and Elliptic Curve Cryptography: Taher ElGamal introduced a public-key encryption and signature approach in the 1980s based on the discrete logarithm problem. Elliptic Curve Cryptography (ECC) later applied elliptic-curve mathematics to public-key operations. ECC can provide comparable security to RSA with much smaller keys, making it useful in devices and systems with limited bandwidth, storage or processing power.

The development of public-key cryptography addressed a central weakness of symmetric systems and made large-scale secure digital communication more practical.

Q5. Advantages and Disadvantages of Asymmetric Cryptography [8 Marks]

Advantages: Public keys can be distributed openly, so users do not need to exchange a shared secret key in advance. Digital signatures support authentication, integrity and non-repudiation. Public-key certificates help bind identities to keys. Public-key techniques also support secure session-key establishment between parties who have never communicated before.

Disadvantages: Public-key operations are generally slower than symmetric encryption and are not efficient for encrypting very large files or continuous data streams. Public-key systems often require longer keys and more computation. Public keys must be authenticated; otherwise, an attacker may substitute a false key. Private keys must be protected carefully, and certificate management, revocation and trust infrastructure add complexity.

Why hybrid cryptosystems are used: Real-world protocols such as TLS combine the strengths of both approaches. Public-key cryptography authenticates the server and/or helps establish shared session keys. After the handshake, fast symmetric authenticated encryption (for example, AES-GCM or ChaCha20-Poly1305) protects the actual application data.

Example: When a browser connects to an HTTPS website, it validates the website's certificate and performs an authenticated key exchange. Both sides derive matching session keys. The browser and server then use symmetric authenticated encryption to protect web pages, passwords and other data. This provides efficient protection without requiring a separate pre-shared secret for every visitor.

Q6. RSA Algorithm and Key Generation [12 Marks]

RSA is a public-key algorithm whose security is associated with the difficulty of factoring a large modulus. It uses one public key for encryption or signature verification and a corresponding private key for decryption or signing.

Key generation steps:

1. Choose two large, distinct prime numbers, p and q.
2. Compute the modulus: $\mathbf{n = p \times q}$. The modulus n is part of both keys.
3. Compute Euler's totient: $\mathbf{\phi(n) = (p - 1)(q - 1)}$.
4. Choose a public exponent e such that $\mathbf{1 < e < \phi(n)}$ and gcd(e, ϕ(n)) = 1. A common choice in practice is e = 65537.
5. Compute the private exponent d, the modular multiplicative inverse of e modulo ϕ(n): $\mathbf{d \times e \equiv 1 \pmod{\phi(n)}}$.
6. Public key is (n, e); private key contains d and the secret prime factors or equivalent protected values.

Encryption: Convert the message to a valid integer m with $0 \le m < n$ using a secure encoding and padding scheme. Calculate the ciphertext: **$c = m^e \bmod n$**.

Decryption: The recipient calculates: **$m = c^d \bmod n$**. In real implementations, RSA encryption uses a padding scheme such as OAEP; raw textbook RSA should not be used.

Small illustrative example (not secure): Let p = 5 and q = 11. Then n = 55 and $\phi(n) = 40$. Choose e = 3, which is relatively prime to 40. The inverse is d = 27 because $3 \times 27 = 81 \equiv 1 \pmod{40}$. If m = 7, then $c = 7^3 \bmod 55 = 13$. Decryption gives 13²■ mod 55 = 7, recovering the original message. These tiny values are only for understanding the mathematics.

Algorithm flowchart

```
START → Select primes p, q → Calculate n = pq and ϕ(n) → Choose e coprime to ϕ(n) →
                Calculate d = e■⁻¹ mod ϕ(n) → Publish (n,e), protect d
                  Sender: encode/pad message → c = m■ mod n → send c
              Receiver: m = c■ mod n → decode → original message → END
```

Why decryption works: Since $ed \equiv 1 \pmod{\phi(n)}$, there is an integer k such that $ed = 1 + k\phi(n)$. Under the RSA number-theoretic conditions, modular exponentiation gives $m^{ed} \equiv m \pmod n$. Therefore, decrypting $c = m^e$ with d recovers m. Practical RSA implementations use carefully specified padding and secure implementation methods.

Q7. RSA Key Sizes and Their Security [10 Marks]

The key size of RSA is the bit length of its modulus n. For example, RSA-2048 uses a modulus that is 2048 bits long. A larger modulus generally makes factoring more difficult and increases the work required by the best known classical factoring algorithms.

RSA security depends on the difficulty of factoring n into its prime factors p and q. If an attacker factors n, they can calculate $\phi(n)$ and derive the private exponent d from the public exponent e. The security of a real RSA deployment also depends on correct padding, key generation, implementation and private-key protection.

RSA modulus	General assessment	Approx. ECC equivalent strength
1024 bits	Obsolete for modern security; should not be used for new systems.	About 160-bit ECC (historical comparison)
2048 bits	Common minimum baseline for many current applications, subject to policy and lifetime needs.	About 224-bit ECC
3072 bits	Stronger classical security and often selected for longer protection horizons.	About 256-bit ECC
4096 bits	Higher classical security margin, with greater computational cost; not automatically necessary for every use.	Roughly 300-bit ECC range; equivalence is approximate

ECC achieves comparable classical security with much shorter keys because its security is based on a different mathematical problem, the elliptic-curve discrete logarithm problem. Equivalence figures are approximate and depend on the standards reference being used.

Quantum-computing threat: Shor's algorithm, if run on a sufficiently large fault-tolerant quantum computer, could efficiently factor RSA moduli and solve the discrete logarithm problems used by many current public-key systems. Increasing RSA key sizes does not provide a practical long-term solution to this threat. Organisations should plan migration to standardised post-quantum cryptography for information that must remain secure for many years.

Q8. Security Considerations and Known Attacks on RSA [12 Marks]

(a) Factoring attacks: RSA's security relies on the difficulty of factoring n = pq. If an attacker can factor n, they can recover the private key. Weak or poorly generated primes, insufficient key sizes, or repeated prime factors can make factoring easier. Use approved cryptographic random-number generators and sufficiently strong key sizes.

(b) Small public exponent and small-message attacks: A small public exponent, such as e = 3, can create risks when textbook RSA is used without secure randomized padding or when the same message is sent to multiple recipients under certain conditions. If the message is small enough that m■ is below n, the ciphertext may reveal the integer root directly. Use RSA-OAEP for encryption and standard, correctly parameterised key choices.

(c) Common modulus attack: If the same RSA modulus n is reused with different public exponents for the same message, and the exponents are relatively prime, an attacker may combine ciphertexts using the extended Euclidean algorithm to recover the message. Each RSA key pair should have its own independently generated modulus; do not reuse an RSA modulus across users.

(d) Timing and other side-channel attacks: An attacker may infer secret information from differences in execution time, power consumption, electromagnetic emissions or cache behaviour during private-key operations. Use constant-time cryptographic implementations, RSA blinding where appropriate, secure hardware when suitable, and avoid custom cryptographic code.

(e) Chosen-ciphertext attacks: In a chosen-ciphertext attack, an attacker can submit selected ciphertexts for decryption and use the responses to learn information about a target ciphertext. Malleable or improperly padded RSA encryption can be vulnerable to such attacks. OAEP (Optimal Asymmetric Encryption Padding) adds randomized encoding and is designed to provide security against chosen-ciphertext attacks when used with RSA in a correctly implemented scheme. Implementations must also avoid revealing distinguishable padding errors.

Padding and RSA digital signatures: Raw RSA is deterministic and mathematically malleable, so it is unsafe for general encryption. RSA encryption should use OAEP. For signatures, the sender hashes the message and signs the digest using the private key with a secure signature encoding, commonly RSA-PSS. The receiver verifies using the public key. A signature provides authenticity and integrity; it does not provide confidentiality.

Good practices for secure RSA deployment:

• Use at least RSA-2048 where RSA is required, and follow current organisational or regulatory guidance for the required protection period.
• Generate keys using a cryptographically secure random-number generator and approved libraries.
• Use RSA-OAEP for encryption and RSA-PSS for new digital-signature designs, with recommended parameters.
• Never use textbook/raw RSA or invent a padding scheme.
• Protect private keys with strong access controls, encryption at rest, hardware security modules where appropriate, and regular key rotation.
• Validate certificates and certificate chains; check validity and revocation status where applicable.
• Use constant-time, maintained cryptographic libraries and keep systems patched.
• Separate encryption and signing keys where the design and standards recommend it, and maintain secure backups and incident-response procedures.

Conclusion

Asymmetric cryptography solves important key-distribution and authentication problems through public/private key pairs, key exchange, digital signatures and certificates. RSA remains an important example, but its security depends on adequate key sizes, secure padding, reliable implementations and careful key management. Modern systems commonly combine public-key methods with fast symmetric encryption, while planning for the future impact of quantum computing.

Suggested references

1. William Stallings, *Cryptography and Network Security: Principles and Practice*.
2. NIST, *Recommendation for Key Management: Part 1 – General* (SP 800-57).

3. NIST, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography* (SP 800-56A).
4. RFC 8017, *PKCS #1: RSA Cryptography Specifications*.
5. NIST Post-Quantum Cryptography project publications.